

به نام خدا



درس سیستم عامل
نیم سال اول ۰۵-۰۴
استاد: دکتر اسدی

دانشکده مهندسی کامپیوتر

تمرین سری چهارم

- پرسش‌های خود را در سامانه CW و تالار مربوط به تمرین مطرح نمایید.
- پاسخ سوالات را تایپ نمایید.
- اسکرین‌شات‌ها، عکس‌ها، فایل‌های مربوط به سوال عملی، گزارش تمرینات عملی و PDF قسمت تئوری را در پوشه با نام به فرمت HWNUM_StudentID1_StudentID2 ذخیره نمایید. سپس آن را zip نمایید و در صفحه درس بارگذاری نمایید. به عنوان مثال یک فایل بارگذاری شده قابل قبول باید دارای فرمت HW1_400123456_403123456.zip باشد.
- هر دانشجو می‌تواند حداکثر سه تمرین را با دو روز تأخیر بدون کاهش نمره ارسال نماید.
- تمرینات عملی و تئوری به صورت گروه‌های دو نفر تحویل داده شود.
- هر دو عضو گروه موظف هستند تمرینات خود را بارگذاری کنند.
- عواقب عدم تطابق بین پاسخ دو عضو گروه برعهده خودشان است.
- تحویل تمرین به صورت انگلیسی مجاز نیست. در صورت تحویل تمرین به صورت انگلیسی (حتی بخشی از تمرین) نمره تمرین موردنظر صفر در نظر گرفته می‌شود.
- در صورت مشاهده تقلب برای بار اول نمره هر دو طرف صفر می‌شود. در صورت تکرار نمره کل تمرینات صفر خواهد شد.
- استفاده از ابزارهایی مانند ChatGPT به منظور ابزار کمک آموزشی مجاز است به شرط آن که به خروجی آن اکتفا نشود.
- توجه شود که پروژه نهایی درس در گروه‌های چهار نفر تحویل گرفته می‌شود.
- سوالات با عنوان اختیاری نمره‌ای ندارند اما جواب دادن به آن‌ها کمک به سزایی در یادگیری درس می‌کند.
- هیچ تمرینی خارج از CW تصحیح نخواهد شد. لذا توصیه می‌شود حداقل یک الی دو ساعت قبل از زمان پایان تمرین برای بارگذاری تمرین اقدام شود.

تمارین تئوری

۱. با توجه به کتابخانه‌ی pthreads به سوالات زیر پاسخ دهید.

(آ) قطعه کد زیر را در نظر بگیرید.

```
1 int balance = 0;
2 void mythread(void *arg) {
3     balance = balance + 200;
4 }
5 thread_t p1, p2;
6 thread_create(&p1, mythread, NULL);
7 thread_create(&p2, mythread, NULL);
8 thread_join(p1);
9 thread_join(p2);
```

وقتی که کد را اجرا کنیم، مقدار نهایی balance چه خواهد بود؟ علت جواب خود را به طور کامل شرح دهید.

(ب) حال فرض کنید که قطعه کد بالا را به قطعه کد پایین تغییر داده‌ایم:

```
1 int balance = 0;
2 void mythread(void *arg) {
3     balance = balance + 200;
4 }
5 thread_t p1, p2;
6 thread_create(&p1, mythread, NULL);
7 thread_join(p1);
8 thread_create(&p2, mythread, NULL);
9 thread_join(p2);
```

وقتی که کد را اجرا کنیم، مقدار نهایی balance چه خواهد بود؟ علت جواب خود را به طور کامل شرح دهید.

(ج) دو قطعه کد را با هم مقایسه کنید و به طور خلاصه علت تفاوت را شرح دهید.

(د) در یک سیستم چند هسته‌ای، اجرای برنامه‌ی چندریسمانی pthreads ممکن است نسبت به سیستم تک‌هسته‌ای نتایج متفاوتی از نظر سرعت و رفتار داشته باشد. توضیح دهید تفاوت میان هم‌زمانی^۱ و موازی‌سازی^۲ در این زمینه چیست. چرا استفاده از چند هسته‌ی پردازشی و افزایش تعداد ریسمان‌ها همیشه باعث افزایش سرعت اجرای برنامه نمی‌شود.

(ه) تکه کد زیر را در نظر بگیرید.

```
1 int *p = NULL;
2 void child(void *arg) {
3     printf("%d\n", *p);
4 }
5 int main(int argc, char *argv[]) {
6     thread_t p1;
7     int x = 3;
8     thread_create(&p1, child, NULL);
9     p = &x;
10    return 0;
11 }
```

چه چیزی چاپ می‌شود و چرا؟

(و) برای اینکه تضمین شود در قسمت (ه) مقدار ۳ چاپ می‌شود، چه راهکاری پیشنهاد می‌کنید تا به کد اضافه شود؟

¹Concurrency

²Parallelism

۲. با توجه به کد زیر، به سوالات پاسخ دهید:

```

1 #include <stdio.h>
2 #include <pthread.h>
3
4 const int x = 100;
5 int counter;
6
7 void* total(void* arg) {
8     for (int i = 1; i <= x; i++)
9         counter++;
10    printf("counter = %d\n", counter);
11    return NULL;
12 }
13
14 int main() {
15     pthread_t t1, t2;
16     counter = 0;
17
18     pthread_create(&t1, NULL, total, NULL);
19     pthread_create(&t2, NULL, total, NULL);
20
21     pthread_join(t1, NULL);
22     pthread_join(t2, NULL);
23
24     return 0;
25 }

```

- (آ) آیا ممکن است هر دو خروجی کمتر از ۲۰۰ باشند؟ پاسخ خود را توضیح دهید.
- (ب) آیا ممکن است یکی از خروجی‌ها کمتر از ۱۰۰ باشد؟ دلیل خود را توضیح دهید.
- (ج) آیا ۱۲۰ و ۱۸۰ از خروجی‌های ممکن این برنامه هستند؟ در صورتی که پاسخ مثبت است، نحوه تولید آن‌ها را توضیح دهید.
- (د) خروجی‌های این برنامه چه مقادیری می‌توانند داشته باشند؟
۳. فرض کنید برنامه‌ای دارید که ترکیبی از کارهای محاسباتی و ورودی/خروجی است. سهم هر نوع کار به صورت زیر است:
- 40% از کارها محاسباتی و وابسته به CPU^۳ هستند (قابل اجرای موازی روی چند هسته)
 - 45% از کارها وابسته به ورودی/خروجی^۴ هستند (در زمان انتظار برای عملیات I/O مسدود می‌شوند)
 - 15% از کارها شامل بخش‌های بحرانی و سریال^۵ هستند که فقط یک ریسه می‌تواند آن‌ها را اجرا کند
- سیستم دارای ۸ هسته پردازشی است. کتابخانه فضای کاربر از مدل many-to-many^۶ با استفاده از LWP^۷ بهره می‌برد، و سیستم عامل از زمان‌بندی پیش‌دستانه^۸ پشتیبانی می‌کند.
- (آ) عملکرد سه مدل نگاشت ریسه‌ها many-to-one، one-to-one و many-to-many را مقایسه کنید. توضیح دهید هر مدل در برابر عملیات‌های CPU و I/O چه رفتاری دارد.

^۳CPU-bound

^۴I/O-bound

^۵Critical / Serial sections

^۶Many-to-many thread mapping

^۷Lightweight Process

^۸Preemptive scheduling

- (ب) با فرض سهم‌های بالا، سرعت بخشی تقریبی هر مدل را در سیستم ۸ هسته‌ای برآورد کنید.
- (ج) اگر کتابخانه فضای کاربر نتواند میزان استفاده واقعی از CPU را تشخیص دهد و تعداد ریشه‌های هسته^۹ کمتر از تعداد هسته‌ها باشد، چه مشکلی پیش می‌آید؟ راهکاری پیشنهاد دهید تا کارایی سیستم حفظ شود.
۴. در حال بهینه‌سازی یک برنامه برای یک پردازنده ۱۶ هسته‌ای هستیم. تحلیل عملکرد نشان می‌دهد که اجرای برنامه به صورت زیر زمان اجرا را مصرف می‌کند:

- پرسش و پاسخ‌های پایگاه داده و پردازش نتایج: 15% (به صورت کاملاً سریالی)
 - عملیات ماتریسی: 60% (کاملاً موازی)
 - اعتبارسنجی داده و عملیات ورودی/خروجی: 25% (فقط 40% از این بخش قابل موازی‌سازی است)
- (آ) درصد زمان اجرای برنامه که به صورت کامل سریال است را حساب کنید.
- (ب) با استفاده از قانون آمثال، حداکثر افزایش سرعت را هنگام اجرای برنامه روی پردازنده ۱۶ هسته‌ای محاسبه کنید.
- (ج) اگر تیم توسعه تصمیم بگیرد بخش اعتبارسنجی داده را بازنویسی کند تا 80% آن قابل موازی‌سازی شود (به جای 40%)، حداکثر شتاب جدید چقدر خواهد بود؟ آیا این بهینه‌سازی با توجه به صرف ۲۰۰ ساعت زمان برنامه‌نویس ارزش دارد؟ (این قسمت جواب واحد ندارد و صرفاً به تحلیل شما نمره داده می‌شود)
- (د) اگر تعداد هسته‌ها به بی‌نهایت نزدیک شود، حداکثر افزایش سرعت ممکن در نسخه اصلی و نسخه بازنویسی شده چقدر خواهد بود؟

۵. یک برنامه از سه بخش تشکیل شده است:

- بخش A (قسمت سریال): 12%
- بخش B (کاملاً موازی‌پذیر): 70%
- بخش C (موازی‌پذیر تا سقف ۸ هسته): 18%

ایجاد و پیوستن ریشه‌ها دارای سربار کوچکی است:

$$H(N) = \frac{0.02}{\log_2(\max(2, N))}$$

- (آ) سرعت بخشی کلی^{۱۰} را برای $N = 2, 4, 8, 16, 32$ محاسبه کنید.
- (ب) کدام مقدار N در عمل بهتر است؟ توضیح دهید.

۶. یک برنامه پردازش ویدئو دارای مراحل خط لوله‌ای^{۱۱} زیر است:

مرحله	شرح	زمان (میلی ثانیه)	موازی‌سازی؟	وابستگی‌ها
A	رمزگشایی ویدئو	۱۲۰	خیر (سریال)	ندارد
B	استخراج فریم	۲۰۰	بله (۸۰٪ موازی)	بعد از A
C	تصحیح رنگ	۱۵۰	بله (۹۵٪ موازی)	بعد از B
D	اعمال فیلتر	۳۰۰	بله (۹۰٪ موازی)	بعد از B
E	رمزگذاری	۱۸۰	خیر (سریال)	بعد از C و D
F	نوشتن خروجی	۸۰	خیر (سریال)	بعد از E

^۹Kernel Threads

^{۱۰}Overall speed-up

^{۱۱}Pipeline

- (آ) زمان اجرای متوالی کامل برای پردازش یک ویدئو را محاسبه کنید.
- (ب) با استفاده از قانون آمدال^{۱۲}، افزایش سرعت نظری^{۱۳} را برای ۲، ۴، ۸ و ۱۶ هسته محاسبه کنید:
- برای هر مرحله موازی‌پذیر به‌طور جداگانه
 - برای کل خط لوله
- (ج) با ۸ هسته در دسترس، یک استراتژی بهینه برای تخصیص رشته^{۱۴} طراحی کنید:
- چند رشته برای هر مرحله نیاز است؟
 - افزایش سرعت واقعی چقدر است؟
 - گلوگاه^{۱۵} سیستم کجاست؟
- (د) با توجه به پاسخ بخش قبل خود، در نظر بگیرید سیستم از مدل رشته یک‌به‌یک^{۱۶} استفاده می‌کند. هر ایجاد رشته 2ms سربار دارد:
- سربار کل را برای طراحی خود محاسبه کنید.
 - افزایش سرعت واقعی را با احتساب سربار مجدداً محاسبه کنید.
۷. (اختیاری) در این سوال می‌خواهیم در ابتدا با فراخوانی‌های سیستمی Non-Blocking آشنا شویم و در ادامه متوجه شویم که ترکیب آن‌ها با ریسمان‌های سطح کاربر^{۱۷} چه ویژگی‌هایی را به همراه دارد.
- (آ) در ابتدا تحقیق کنید که فراخوانی‌های سیستمی `epoll` و `io_uring` چه هستند و چه تفاوت‌هایی با فراخوانی‌هایی نظیر `read` دارند.
- (ب) فراخوانی‌های سیستمی `io_uring` چه برتری نسبت به فراخوانی‌های سیستمی `epoll` دارند؟
- (ج) فرض کنید که یک برنامه در اختیار دارید که از مدل `Many-To-One` استفاده می‌کند. در صورتی که یکی از ریسمان‌های سطح کاربر این برنامه، از فراخوانی سیستمی `read` استفاده کند چه اتفاقی می‌افتد؟ آیا بقیه ریسمان‌های سطح کاربر می‌توانند به کار خود ادامه دهند؟
- (د) توضیح دهید که چگونه می‌توانستیم به کمک فراخوانی‌های سیستمی Non-Blocking مانند `io_uring` یا `epoll` مشکل گفته شده در بالا را حل کنیم.
- (ه) در برخی از زبان‌های برنامه‌نویسی مانند `C#` یا `Python` یا `Rust` مفهوم `Task` یا `Coroutine` وجود دارد که معمولاً به همراه کلیدواژه‌های `async/await` استفاده می‌شوند. توضیح دهید که در پشت صحنه و زمان استفاده از این مفاهیم چه اتفاقی در حال رخ دادن است.
- (و) توضیح دهید که چرا استفاده از مفاهیم مربوط به ریسمان‌های سطح کاربر همراه با فراخوانی‌های سیستمی Non-Blocking می‌تواند در برنامه‌هایی که بارکاری زیاد از نوع I/O مخصوصاً شبکه محور دارند مفید واقع شود. آیا استفاده از `async/await` در برنامه‌های `CPU Bound` می‌تواند مفید واقع شود؟
- (ز) تفاوت `Goroutine`‌ها در زبان برنامه‌نویسی `Golang` با رشته‌های هسته در چیست؟ حال با توجه به این تفاوت، توضیح دهید `runtime` زبان `Golang` چگونه با استفاده از مدل رشته چندبه‌چند زمان‌بندی `Goroutine`‌ها را مدیریت می‌کند.

¹²Amdahl's Law¹³Theoretical Speedup¹⁴Thread¹⁵Bottleneck¹⁶One-to-One Threading Model¹⁷User Threads

تمارین عملی

۱. XV6 OS

برای این تمرین از کدی که در سامانه CW قرار گرفته است استفاده کنید!

در این تمرین قصد داریم تا مکانیسم context switch را برای ریشه‌های سطح کاربر^{۱۸} بررسی کنیم و آن را پیاده‌سازی کنیم. وظیفه شما در این تمرین پیاده‌سازی کدی است که ریشه‌ها را تشکیل دهد، آن‌ها را اجرا کند و ثبات‌های آن‌ها را ذخیره و بازیابی کند. در ادامه به بررسی مراحل این تمرین می‌پردازیم.

- در ابتدا نیاز دارید که یک struct با نام context داشته باشید که ثبات‌های یک ریشه را شامل شود. برای نحوه پیاده‌سازی آن می‌توانید به فایل kernel/proc.h مراجعه کنید.
- حال باید struct ریشه را تعریف کنید که به صورت زیر تعریف می‌شود:

```
1 struct thread {
2   char    stack[STACK_SIZE]; // the thread's stack
3   int state;                  // FREE, RUNNING, RUNNABLE
4   struct context context;     // swtch() here to run process
5 };
```

- در مرحله بعدی باید تابع thread_init را تعریف کنید که یک ریشه را به تابع main اختصاص می‌دهد و آن را در وضعیت اجرا قرار می‌دهد.
- همچنین به یک تابع thread_create نیاز دارید تا بر روی همه ریشه‌ها حلقه بزند و ریشه‌ای که در وضعیت آزاد قرار دارد را پیدا کند و فضایی را به context و پشته آن اختصاص دهد و آن را در حالت آماده اجرا قرار دهد.
- مهمترین تابع این تمرین، تابع thread_schedule است که وظیفه زمان‌بندی را به عهده دارد. به طور خلاصه این تابع باید بر روی همه ریشه‌ها حلقه بزند و اگر ریشه‌ای آماده اجرا دید، تغییر زمینه^{۱۹} کند و آن ریشه را اجرا کند. همچنین اگر ریشه‌ای آماده اجرا وجود نداشت، این برنامه باید به اتمام برسد.
- همچنین به یک تابع کمکی thread_yield نیاز دارید که وضعیت ریشه را در حالت آماده اجرا قرار دهد و سپس تابع thread_schedulde را صدا کند.
- در نهایت شما باید یک برنامه بنویسید که شامل حداقل سه ریشه است و این سه ریشه باید خروجی زیر را نمایش دهند و سپس به پایان برسند. دقت کنید که باید در پایان هر چاپ کردن، وضعیت ریشه خود را در حالت آماده اجرا قرار دهید تا در مرحله بعدی نیز بتواند اجرا شود.

```
1 $ uthread
2 thread_a started
3 thread_b started
4 thread_c started
5 thread_c 0
6 thread_a 0
7 thread_b 0
8 thread_c 1
9 thread_a 1
10 thread_b 1
11 ...
12 thread_b 98
13 thread_c 99
14 thread_a 99
15 thread_b 99
```

^{۱۸}user-level thread

^{۱۹}context switch

```

16 thread_c: exit after 100
17 thread_a: exit after 100
18 thread_b: exit after 100
19 thread_schedule: no runnable threads

```

۲. Docker

توصیه می‌شود این تمرین را در یک ماشین مجازی انجام دهید تا مشکلات احتمالی سیستم شما را دچار مشکل نکند. در این تمرین قصد داریم در ادامه تمرین قبل، نکات مربوط به فضای نام در داکر را بررسی و پروژه را کامل کنیم.

گام‌های تمرین

آ. آخرین نسخه پروژه را دریافت کنید و به تگ t2 بروید.

```
1 git checkout t2
```

ب. یک پوسته را داخل محیط کانتینر اجرا کنید.

```
1 ./zocker run --name test-container 'sh'
```

حال به واسطه دستور ps لیست تمام پردازش‌های موجود در کانتینر را مشاهده کنید.

سوال: چرا علی‌رغم اجرای کانتینر در یک فضای نام PID جداگانه، خروجی دستور مطابق انتظار نیست و لیست تمام پردازش‌های موجود در host نشان داده می‌شود؟ (راهنمایی: در رابطه با عملکرد دستور ps جست‌وجو کنید و بررسی کنید به چه روشی اطلاعات مربوط به پردازش‌ها را بدست می‌آورد)

ج. با بررسی این لینک و این لینک، تابع run_container را به نحوی تغییر دهید که دستور ps در داخل کانتینر نتیجه قابل انتظاری داشته باشد.

د. پس از انجام بخش قبلی و اجرای درست دستور ps، امکان دارد عملکرد سیستم دچار مشکل شود. برای مثال اجرای دستور df خطایی مشابه زیر تولید خواهد کرد.

```
1 df: cannot read table of mounted file systems: No such file or directory
```

برای رفع مشکل پیش‌آمده می‌توانید از دستور زیر استفاده کنید:

```
1 sudo umount /proc
```

سوال: چرا با وجود جدا کردن فضای نام mount، سایر های‌lrmount سیستم دچار مشکل شدند؟ (راهنمایی: در رابطه با سلسله‌مراتب^{۲۰} و انتشار^{۲۱} فضای نام‌های mount جست‌وجو کنید)

ه. حال با بررسی پرچم‌های فراخوان سیستم mount(2)، کد را به نحوی تغییر دهید که با حفظ خروجی مورد انتظار از بروز این مشکل جلوگیری کند.

و. یک فضای نام دیگر برای پردازش‌ها، فضای نام UTS است. کد را به نحوی تغییر دهید که برای هر کانتینر این فضای نام به صورت جداگانه اختصاص پیدا کند.

ز. با استفاده از فراخوان سیستمی sethostname(2) کد را به نحوی تغییر دهید که با اجرای دستور hostname در پوسته داخل یک کانتینر، به جای نام ماشین میزبان، نام کانتینر که در هنگام اجرا مشخص شده است، در خروجی نمایش داده شود.

ح. با استفاده از پرچم‌های مناسب فضای نام زمان و کاربر کانتینر را به صورت جداگانه هنگام اجرای کانتینر اختصاص دهید.

²⁰hierarchy

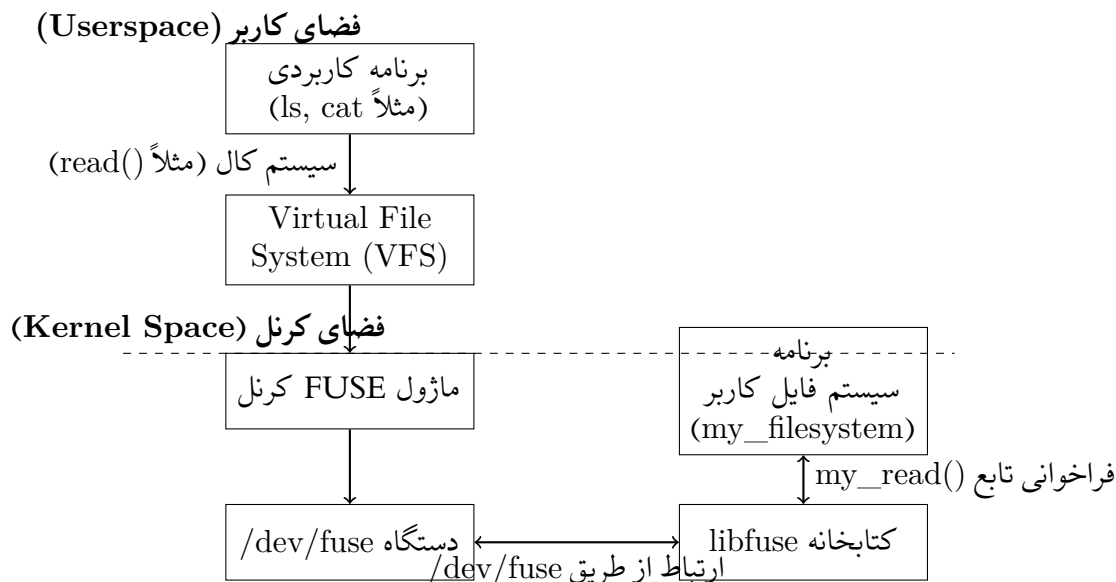
²¹propagation

موارد لازم جهت تحویل

- پاسخ به دو سوال.
- خروجی قابل انتظار بخش های ج، ه و و.
- کدهای تغییر یافته یا آدرس مخزن حاوی کدهای تغییر یافته.

۳. توسعه ماژول هسته

سامانه مدیریت پرونده در سطح کاربر (FUSE) در این تمرین می‌خواهیم یک سامانه مدیریت پرونده در سطح کاربر بسازیم. به این سامانه‌های در دنیای لینوکس FUSE یا File-system in User Space گفته می‌شود.



مزایای استفاده و توسعه سامانه‌های مدیریت پرونده سطح کاربر

امنیت و پایداری: از آنجایی که منطق سیستم فایل در فضای کاربر اجرا می‌شود، یک خطا یا خرابی در برنامه FUSE باعث خرابی کل سیستم عامل نمی‌شود. تنها پردازش مربوطه از بین می‌رود.

سهولت توسعه: توسعه‌دهندگان می‌توانند سیستم فایل خود را با استفاده از زبان‌های برنامه‌نویسی معمولی مانند C و C++ و Python و Go و Rust و... بنویسند. نیاز به دانش برنامه‌نویسی هسته یا کامپایل مجدد هسته نیست.

انعطاف‌پذیری: FUSE برای پیاده‌سازی طیف وسیعی از سیستم‌های فایل که داده‌ها را به صورت خطی روی یک دیسک بلوک‌بندی شده ذخیره نمی‌کنند، ایده‌آل است. شما می‌توانید از هر منبع داده‌ای به عنوان پشتیبان استفاده کنید.

قابل استفاده توسط کاربران عادی: هر کاربری می‌تواند سیستم فایل FUSE خود را بدون دسترسی روت mount کند (با استفاده از آپشن allow_other یا با تنظیم مجوزهای مناسب).

هدف تمرین

- آشنایی با مفهوم FUSE
- درک راحت‌تر مفاهیم پرونده و پوشه
- پیاده‌سازی اجزای منطقی سامانه مدیریت پرونده

● پیاده‌سازی یک سامانه مدیریت پرونده تحت رابط کاربری خط فرمان به زبان C

مراحل تمرین

- طراحی و پیاده‌سازی سامانه‌ای که بتواند هر بازه دلخواه از یک فایل دلخواه مانند `filesys.db` را بخواند و در صورتی که وجود نداشت یا ابعاد آن مشکل داشت آن را بسازد. حجم این فایل ثابت است و عملاً نماینده قطعه‌ای از دیسک است که با سامانه مدیریت پرونده ما مدیریت می‌شود.
- طراحی یک داده ساختار برای نگهداری اطلاعات کلی سیستم مدیریت پرونده. توجه کنید که این داده ساختار به مرور زمان کامل می‌شود و در ابتدای `filesys.db` قرار می‌گیرد. همچنین فیلد اول آن باید متغیری از نوع `int32` با نام `magic` باشد که تنها مقدار معتبر آن `0xDEADBEEF` است و این موضوع در هنگام شروع سامانه بررسی می‌شود. همچنین فیلد بعدی آن یک `int32` دیگر برای ذخیره نسخه سامانه است.
- طراحی داده ساختارهای مناسب برای توصیف و نگهداری پرونده‌ها مبتنی بر نامشان. برای این تمرین فرض کنید تنها یک پوشه ریشه داریم و همه پرونده‌ها به صورت مستقیم درون آن قرار دارند و همچنین درون نامشان / وجود ندارد. داده‌های هر فایل را می‌توانید به صورت: `name` و `type` و `permission` و `data` در نظر بگیرید و همینطور برای این تمرین می‌توانید فرض کنید داده ساختار نگهداری پرونده‌ها یک لیست پیوندی است.
- به طور مشابه با بخش قبل یک داده ساختار برای مدیریت فضاهای خالی طراحی کنید. برای این تمرین کافی است صرفاً یک عدد درون `metadata` نگهداری کنید که آدرس آخرین بلوک استفاده شده را نگهداری کند.
- دستوراتی که در این تمرین باید پیاده کنید:
 - `open(filename, flags)`: باز کردن و یا ساختن یک پرونده با نام `filename` flags. یک `int32` است که یک مجموعه پرچم مانند `CREATE` و `WRITE` را شامل می‌شود.
 - `read(pos, n_bytes)`: خواندن حداکثر `n` بایت با شروع از `pos`.
 - `write(pos, n_bytes)`: نوشتن `n` بایت با شروع از `pos`.
 - `shrink(new_size)`: نگهداشتن * تا `new_size` از پرونده باز شده و حذف بقیه آن.
 - `get_file_stats()`: گرفتن اندازه پرونده.
 - `rm()`: بستن و حذف پرونده باز شده.
 - `close()`: بستن پرونده باز شده.
 - `get_fs_stats()`: حجم مصرف شده و خالی و تعداد پرونده‌ها را نمایش می‌دهد.